```
'use strict';
/**
 * Port serveur de "MDT Sniper Pro" (Pine v5) : logique de signal + gestion de tr
 * Entrée : bougies CLÔTURÉES, une par une, via onCandle(bougie, { ema1, ema2 }).
 * Sortie : liste d'événements JSON, même format que les alertes TradingView
 * (ACHAT, VENTE, TP1_HIT, TP2_HIT, TP3_HIT, BE_HIT, SL_HIT).
 * Aucune dépendance.
 */

const DEFAULTS = {
  capital: 1000, riskPct: 0.5, contractSize: 100,
  slBuf: 0.3, slLook: 6, minRiskATR: 0.4, maxRiskATR: 3.5,
  tp1R: 0.5, tp2R: 1.5, tp3R: 3.0,
  useDaily: true, dailyLimit: 2.0,
  atrLen: 14, trendMult: 2.0,
  profile: 'Sniper A+', // 'Starter' | 'Intraday Clean' | 'Sniper A+'
  useRetest: true, retestLook: 10, cooldown: 15,
  sweepLen: 20, microLen: 2, dispAtr: 0.8,
  volLen: 20, volMult: 1.4, maxZones: 6, maxZoneATR: 3.0,
  secret: '', symbol: 'XAUUSD',
};

const r2 = (x) => Math.round(x * 100) / 100;

class MdtEngine {
  constructor(cfg = {}) {
    this.c = { ...DEFAULTS, ...cfg };
    const p = this.c.profile;
    this.reqScore = p === 'Starter' ? 2 : p === 'Intraday Clean' ? 4 : 5;
    this.reqZone = this.reqHtf = p === 'Sniper A+';

    this.bars = []; this.i = -1;
    this.trs = []; this.atr = null;
    this.st = null; this.td = 1;
    this.microHigh = null; this.microLow = null;
    this.bull = []; this.bear = [];
    this.last = {};            // dernier index de chaque condition (équivalent b
    this.armL = null; this.armS = null; // { level, bar }
    this.lastSig = -1e9;
    this.trade = null;
    this.dayR = 0; this.day = null;
  }
```

```
_ev(action, extra) {
  return { secret: this.c.secret, action, symbol: this.c.symbol, ...extra };
}

/** À appeler après le rejeu de l'historique : évite de rouvrir de vieux trades
finishWarmup() {
  this.trade = null; this.armL = null; this.armS = null; this.dayR = 0;
}

onCandle(k, htf = {}) {
  const c = this.c, B = this.bars, ev = [];
  const prev = B[B.length - 1];
  B.push(k);
  if (B.length > 600) B.shift();
  const n = B.length, i = ++this.i;

  // ── ATR (RMA)
  const tr = prev
    ? Math.max(k.high - k.low, Math.abs(k.high - prev.close), Math.abs(k.low -
    : k.high - k.low;
  if (this.atr == null) {
    this.trs.push(tr);
    if (this.trs.length === c.atrLen) this.atr = this.trs.reduce((a, b) => a +
  } else {
    this.atr = (this.atr * (c.atrLen - 1) + tr) / c.atrLen;
  }
  const atr = this.atr;
  if (atr == null || n < 30) return ev; // phase de chauffe

  // ── Jour (UTC) : remise à zéro du R journalier
  const t = new Date(k.time);
  const day = t.toISOString().slice(0, 10);
  if (day !== this.day) { this.day = day; this.dayR = 0; }

  // ── Ligne de tendance MDT
  const upS = k.low - atr * c.trendMult, dnS = k.high + atr * c.trendMult;
  const prevTd = this.td;
  if (this.st == null) this.st = upS;
  if (prevTd === 1) {
    if (upS > this.st) this.st = upS;
    if (k.close < this.st) { this.td = -1; this.st = dnS; }
  }
  if (prevTd === -1) {
    if (dnS < this.st) this.st = dnS;
    if (k.close > this.st) { this.td = 1; this.st = upS; }
  }
```

```
const trendUp = this.td === 1;
const flipUp = this.td === 1 && prevTd === -1;
const flipDn = this.td === -1 && prevTd === 1;

// ── Micro-structure (pivots)
const pmh = this.microHigh, pml = this.microLow;
const L = c.microLen;
const cand = B[n - 1 - L];
let isH = true, isL = true;
for (let j = 1; j <= L; j++) {
  const left = B[n - 1 - L - j], right = B[n - 1 - L + j];
  if (!(cand.high > left.high) || !(cand.high >= right.high)) isH = false;
  if (!(cand.low < left.low) || !(cand.low <= right.low)) isL = false;
}
if (isH) this.microHigh = cand.high;
if (isL) this.microLow = cand.low;

const body = Math.abs(k.close - k.open);
const dispUp = k.close > k.open && body >= atr * c.dispAtr;
const dispDn = k.close < k.open && body >= atr * c.dispAtr;
const microBreakUp = this.microHigh != null && pmh != null &&
  k.close > this.microHigh && prev.close <= pmh && dispUp;
const microBreakDn = this.microLow != null && pml != null &&
  k.close < this.microLow && prev.close >= pml && dispDn;

// ── Sweeps de liquidité
let llv = Infinity, hhv = -Infinity;
for (let j = 1; j <= c.sweepLen; j++) {
  const b = B[n - 1 - j];
  llv = Math.min(llv, b.low); hhv = Math.max(hhv, b.high);
}
const sweepBull = k.low < llv && k.close > llv;
const sweepBear = k.high > hhv && k.close < hhv;

// ── Volume
let vs = 0;
for (let j = 0; j < c.volLen; j++) vs += B[n - 1 - j].volume || 0;
const volAvg = vs / c.volLen;
const hvb = (k.volume || 0) > 0 ? k.volume > volAvg * c.volMult : true;

// ── Filtre tendance de fond H1/H4 (EMA200 fournies par l'appelant)
const { ema1, ema2 } = htf;
const htfUp = ema1 != null && ema2 != null && k.close > ema1 && k.close > ema
const htfDn = ema1 != null && ema2 != null && k.close < ema1 && k.close < ema

// ── Zones OB : contact (avant nettoyage), nettoyage, ajout
```

```
const touch = (zones) => {
  for (let j = zones.length - 1; j >= 0; j--) {
    if (k.low <= zones[j].top && k.high >= zones[j].bot) return true;
  }
  return false;
};
const tbHit = touch(this.bull), tsHit = touch(this.bear);
this.bull = this.bull.filter((z) => !(k.close < z.bot));
this.bear = this.bear.filter((z) => !(k.close > z.top));
const o2 = B[n - 3];
if ((o2.high - o2.low) <= c.maxZoneATR * atr) {
  if (microBreakUp) {
    this.bull.push({ top: o2.high, bot: o2.low });
    if (this.bull.length > c.maxZones) this.bull.shift();
  }
  if (microBreakDn) {
    this.bear.push({ top: o2.high, bot: o2.low });
    if (this.bear.length > c.maxZones) this.bear.shift();
  }
}

// ── Risque (SL sur pivot swing)
let hiL = -Infinity, loL = Infinity;
for (let j = 0; j < c.slLook; j++) {
  const b = B[n - 1 - j];
  hiL = Math.max(hiL, b.high); loL = Math.min(loL, b.low);
}
const slB = loL - c.slBuf * atr, slS = hiL + c.slBuf * atr;
const okRisk = (r) => r >= c.minRiskATR * atr && r <= c.maxRiskATR * atr;
const cB7 = okRisk(k.close - slB), cS7 = okRisk(slS - k.close);

// ── Score de confluence (7 points)
const Lst = this.last;
if (tbHit) Lst.tb = i;
if (tsHit) Lst.ts = i;
if (sweepBull) Lst.swB = i;
if (sweepBear) Lst.swS = i;
if (microBreakUp) Lst.mbU = i;
if (microBreakDn) Lst.mbD = i;
if (flipUp) Lst.fU = i;
if (flipDn) Lst.fD = i;
if (hvb) Lst.hv = i;
const rec = (key, w) => i - (Lst[key] ?? -1e9) <= w;
const W = 6;

const cB = [!this.reqHtf || htfUp, rec('tb', W), rec('swB', W), rec('mbU', W)
```

```
  trendUp && rec('fU', 15), rec('hv', W), cB7];
const cS = [!this.reqHtf || htfDn, rec('ts', W), rec('swS', W), rec('mbD', W)
  !trendUp && rec('fD', 15), rec('hv', W), cS7];
const scoreB = cB.filter(Boolean).length, scoreS = cS.filter(Boolean).length;

// ── Session (UTC, lun-ven)
const dow = t.getUTCDay(), mins = t.getUTCHours() * 60 + t.getUTCMinutes();
const wk = dow >= 1 && dow <= 5;
const inLondon = wk && mins >= 420 && mins < 660;
const inNY = wk && mins >= 750 && mins < 1020;
const sess = c.profile === 'Starter' || inLondon || inNY;

// ── Setup + retest strict
const setupL = flipUp && cB7 && scoreB >= this.reqScore && (!this.reqZone ||
const setupS = flipDn && cS7 && scoreS >= this.reqScore && (!this.reqZone ||
if (setupL) this.armL = { level: this.microHigh, bar: i };
if (setupS) this.armS = { level: this.microLow, bar: i };
const a = this.armL, s = this.armS;
const retestL = !!a && a.level != null && i > a.bar && k.low <= a.level && k.
const retestS = !!s && s.level != null && i > s.bar && k.high >= s.level && k
const expL = !!a && i - a.bar > c.retestLook;
const expS = !!s && i - s.bar > c.retestLook;
const can = i - this.lastSig > c.cooldown;
const sigBuy = can && (c.useRetest ? retestL : setupL);
const sigSell = can && (c.useRetest ? retestS : setupS);
if (sigBuy) { this.armL = null; this.lastSig = i; }
if (sigSell) { this.armS = null; this.lastSig = i; }
if (expL) this.armL = null;
if (expS) this.armS = null;

// ── Ouverture
const dayBlocked = c.useDaily && this.dayR < 0 && (-this.dayR * c.riskPct) >=
const openBuy = sigBuy && !this.trade && !dayBlocked;
const openSell = sigSell && !this.trade && !openBuy && !dayBlocked;
if (openBuy || openSell) {
  const dir = openBuy ? 1 : -1;
  const entry = k.close, sl = dir === 1 ? slB : slS;
  const risk = Math.abs(entry - sl);
  const raw = risk > 0 ? (c.capital * c.riskPct / 100) / (risk * c.contractSi
  const lot = Math.max(0.01, Math.round(raw * 100) / 100);
  const tp = (r) => entry + dir * r * risk;
  this.trade = {
    dir, entry, sl, risk, tp1: tp(c.tp1R), tp2: tp(c.tp2R), tp3: tp(c.tp3R),
    tp1Done: false, tp2Done: false, bar: i,
  };
  const T = this.trade;
```

```
    ev.push(this._ev(dir === 1 ? 'ACHAT' : 'VENTE', {
      entry: r2(entry), sl: r2(sl), tp1: r2(T.tp1), tp2: r2(T.tp2), tp3: r2(T.t
      lot: lot.toFixed(2), risk_r: 1.0, score: dir === 1 ? scoreB : scoreS,
    }));
  }

  // ── Gestion du trade en cours (TP1 50% + BE, TP2 25%, TP3 runner 25%)
  const T = this.trade;
  if (T && T.bar < i) {
    const up = T.dir === 1;
    const hit = (lv) => (up ? k.high >= lv : k.low <= lv);
    const hTP1 = hit(T.tp1), hTP2 = hit(T.tp2), hTP3 = hit(T.tp3);
    const hSL = up ? k.low <= T.sl : k.high >= T.sl;
    const price = r2(k.close);

    if (hTP1 && !T.tp1Done) {
      T.tp1Done = true;
      T.sl = T.entry + (up ? 0.30 : -0.30);
      ev.push(this._ev('TP1_HIT', { price, partial_pct: 50, move_sl_to_be: true
    }
    if (hTP2 && !T.tp2Done) {
      T.tp2Done = true;
      ev.push(this._ev('TP2_HIT', { price, partial_pct: 25 }));
    }
    if (hTP3) {
      const g = 0.5 * c.tp1R + 0.25 * c.tp2R + 0.25 * c.tp3R;
      this.dayR += g; this.trade = null;
      ev.push(this._ev('TP3_HIT', { price, close_all: true, total_r: g }));
    } else if (hSL) {
      if (T.tp2Done) {
        const g = 0.5 * c.tp1R + 0.25 * c.tp2R;
        this.dayR += g;
        ev.push(this._ev('BE_HIT', { price, close_all: true, locked_r: g }));
      } else if (T.tp1Done) {
        const g = 0.5 * c.tp1R;
        this.dayR += g;
        ev.push(this._ev('BE_HIT', { price, close_all: true, locked_r: g }));
      } else {
        this.dayR -= 1;
        ev.push(this._ev('SL_HIT', { price, close_all: true, loss_r: -1.0 }));
      }
      this.trade = null;
    }
  }
  return ev;
}
```

```
}

module.exports = { MdtEngine, DEFAULTS };
```